MOVATE | DEVOPS ENGINEERING | CLOUD OPERATIONS

GCP Service Account Key Rotation Automation

Cloud Run Job-Based Secure Key Lifecycle Management

DEVELOPED BY THE DEVOPS TEAM @ MOVATE

ROLE	NAME	CONTACT
Team Lead	**Vignesh Nagachalavelavan**	Vignesh.Nagachalavelavan01@movate.com
Engineer	**Shiv Kumar Verma**	ShivKumar.Verma@movate.com
Engineer	**Srigopinath Angamuthu Raja**	Srigopinath.AngamuthuRaja@movate.com
DevOps Engineers	**Movate DevOps Team**	—

Table of Contents

1 Executive Summary

This solution automates the **detection, rotation, validation, and reporting** of Google Cloud service account keys across multiple GCP projects. It eliminates manual key rotation risks, expiry-related outages, and security vulnerabilities caused by long-lived credentials.

Key Outcomes

- Secure
- Automated
- Scalable
- Auditable

Outcome	Description
Secure	Private key never leaves your environment — only the public X.509 certificate is sent to GCP
Automated	Fully scheduled lifecycle — scan, rotate, validate, and report without human intervention
Scalable	Multi-project support across any number of GCP projects in a single run
Auditable	Every rotation is logged in GCS, Cloud Audit Logs, and delivered via email with an Excel attachment

> **Zero-downtime rotation** is ensured through overlapping key validity — the old key remains active in GCP IAM until the application team updates their credentials.

2 Business Problem

Challenge	Impact
Manual key rotation	Human error, missed expiry deadlines
No centralised visibility	Poor governance, no audit trail across projects
Expired keys	Application downtime and failed API calls
Long-lived credentials	Increased attack surface and security risk

3 Solution Overview

A **Cloud Run Job** executes on a schedule to perform the full key lifecycle automatically:

1. **Scan** all configured GCP projects for user-managed service account keys
2. **Identify** keys expiring within a configurable threshold (default: 14 days)
3. **Rotate** keys securely using RSA-2048 asymmetric cryptography

4. **Store** new credentials in GCS with controlled, IAM-gated access
5. **Validate** that the new key works, with retry logic for GCP propagation delay
6. **Report** results via a colour-coded HTML email and a full Excel attachment

Two Operating Modes

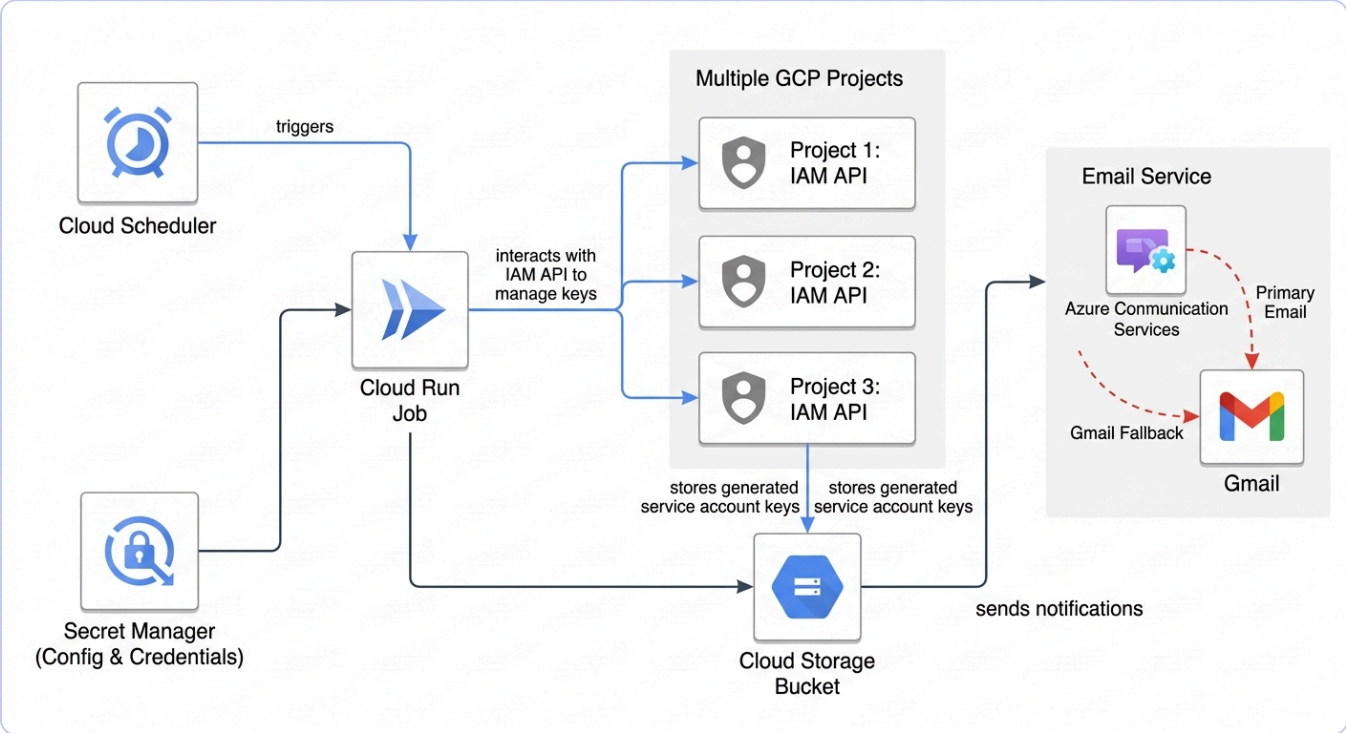
Mode	Setting	Behaviour
Rotate Mode	`ENABLE_ROTATION=true`	Full rotation — scans and rotates all expiring keys. Default.
Scan Mode	`ENABLE_ROTATION=false`	Audit only — reports expiry status without making any changes.

4 High-Level Architecture (HLD)

Components

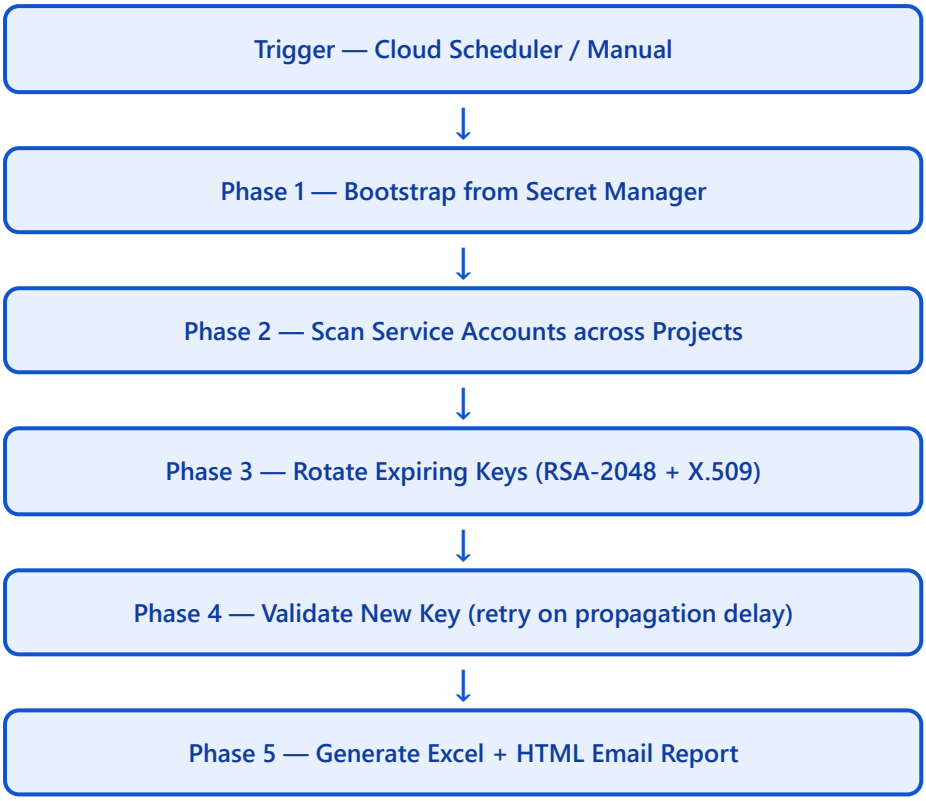
Component	Role
Cloud Run Job	Execution engine — runs the full rotation workflow on schedule
Secret Manager	Stores all configuration and the main service account credentials
IAM API	Key listing, creation, and public certificate upload
GCS Bucket	Persistent, access-controlled storage for rotated key JSON files
Email Service	ACS Email (primary) + Gmail SMTP (fallback) for report delivery
Cloud Scheduler	Triggers the Cloud Run Job on a cron schedule (e.g. weekly)

Architecture Diagram



5 Low-Level Design (LLD)

Execution Flow



Detailed Phase Breakdown

Phase	Actions
Bootstrap	Cloud Run attached SA fetches main SA credentials from Secret Manager; all config loaded at runtime
Scan	Lists all service accounts per project; evaluates the most recently created user-managed key per SA
Rotate	Generates RSA-2048 key pair offline; wraps public key in X.509 cert; uploads cert to GCP IAM; stores JSON in GCS
Validate	Obtains an access token using the new key; retries up to 3× with 10s delay for GCP propagation (~30s)
Report	Builds colour-coded Excel report and HTML email; sends via ACS with Gmail SMTP fallback on 429

> **Private key security:** Only the X.509 public certificate is uploaded to GCP. The private key is assembled into the JSON credential and stored directly in GCS — it never travels to GCP's API.

6 Key Technical Highlights

Security

- Private key generated **offline** — never sent to GCP over the network
- Only the X.509 public certificate is uploaded via `upload_public_key()`
- All sensitive configuration stored in Secret Manager — zero hardcoded credentials

Flexibility

- **Rotate Mode** — active rotation of expiring keys on schedule
- **Scan Mode** — audit-only reporting without any mutations

Reliability

- Per-service-account error isolation — one failure does not block others
- Key validation with 3 retries + 10-second delays to handle GCP's ~30s propagation window
- Dual email delivery: ACS primary → Gmail SMTP fallback on rate-limit (429)

Reporting

- Colour-coded HTML email with metrics summary card and key details table
- Full Excel attachment: 10 columns, auto-filter, row-level colour coding by status
- Stakeholder-friendly format for governance reviews and compliance evidence

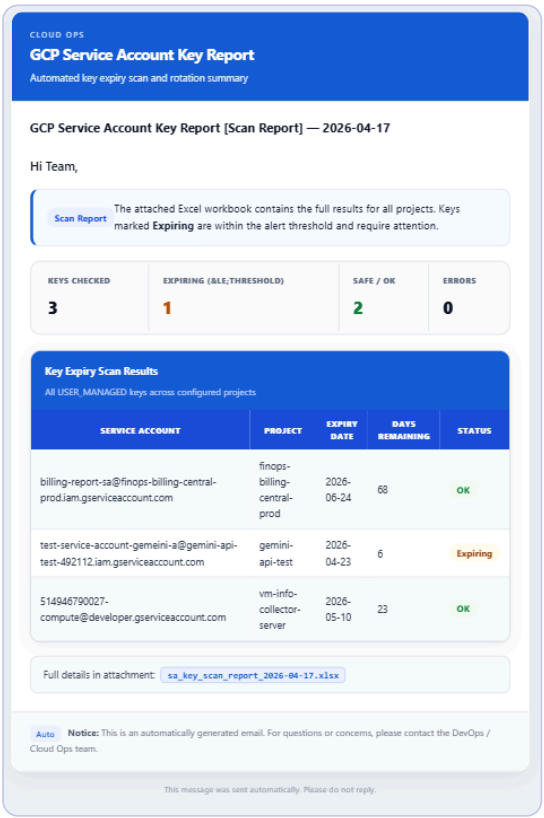


Figure 1 — Sample HTML Email Report: colour-coded status table with metrics summary card

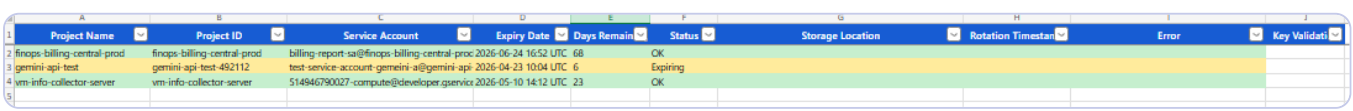


Figure 2 — Sample Excel Attachment: 10-column report with auto-filter and row-level colour coding

7 Key Rotation Policy & Behavior

Rotation Policy

- Keys are rotated **14 days before expiry** (configurable via Secret Manager)
- Controlled by: `EXPIRY_THRESHOLD_DAYS = 14`

Behavior in GCP IAM

- A **new key is created** in GCP IAM via public cert upload
- The **existing (old) key remains active** — it is NOT deleted automatically

> This ensures **zero downtime** — applications continue using the old key until the team distributes and applies the new one. Old and new keys coexist in GCP IAM during the transition window.

Behavior in GCS Bucket

- Only the **latest rotated key is stored** per service account path
- All previous key files for a SA are **automatically deleted** when a new key is saved

Summary

Aspect	Behavior
Rotation trigger	14 days before expiry (configurable)
GCP IAM keys after rotation	Old + New coexist until old key expires or is deleted
GCS storage	Latest key only — older files automatically deleted
Application downtime	None — old key remains valid during transition

8 Technology Stack

Layer	Technology
Compute	Google Cloud Run Jobs
Scheduler	Google Cloud Scheduler
Secrets	Google Secret Manager
Storage	Google Cloud Storage (GCS)
IAM	GCP IAM v1 REST API
Cryptography	Python `cryptography` library (RSA-2048, X.509 self-signed cert)
Email (primary)	Azure Communication Services (ACS) Email
Email (fallback)	Gmail SMTP — `smtp.gmail.com:587`, STARTTLS
Reporting	`openpyxl` (Excel), HTML (email template)
Language	Python 3.11

9 Code Structure

```
gcp-sa-key-rotation/
├── main.py                 # Entry point: orchestration, report, email dispatch
├── requirements.txt        # Python dependencies
├── .env                    # Bootstrap env vars (not committed)
└── src/
    ├── config.py           # 2-step Secret Manager bootstrap + AppConfig dataclass
    ├── gcp_client.py       # Thin GCP IAM v1 API wrapper
    ├── key_manager.py      # RSA generation, X.509 certs, JSON credential assembly
    ├── rotator.py          # Core logic: scan, rotate, validate, error isolation
    ├── storage.py          # GCSStorage / LocalStorage abstraction
    ├── acs_email_client.py # ACS primary + Gmail SMTP fallback
    └── email_template.py   # HTML email builder: metrics card + status table
```

Module	Responsibility
`main.py`	Top-level orchestration, exit codes, report generation, email dispatch
`config.py`	2-step Secret Manager bootstrap, AppConfig dataclass
`gcp_client.py`	Stateless IAM API wrapper — list SAs, list keys, upload public cert
`key_manager.py`	Offline cryptography — RSA pair, X.509 wrapping, JSON credential assembly

Module	Responsibility
`rotator.py`	Business logic — expiry check, rotation, validation, per-SA error isolation
`storage.py`	GCSStorage (auto-cleans old keys) and LocalStorage
`acs_email_client.py`	ACS with 429 fallback to Gmail SMTP
`email_template.py`	HTML email with metrics card and colour-coded status table

10 Configuration

Bootstrap Environment Variables

Variable	Required	Description
`SECRET_MANAGER_PROJECT_ID`	Yes	GCP project that owns all the secrets
`SERVICE_ACCOUNT_SECRET_ID`	Yes	Secret ID containing the main SA JSON credentials
`SECRET_MANAGER_VERSION`	No	Secret version to fetch (default: `latest`)
`LOG_LEVEL`	No	Logging verbosity: DEBUG / INFO / WARNING (default: INFO)

Secrets Loaded from Secret Manager

Secret ID	Default	Description
`GCP_PROJECTS`	—	Comma-separated list of project IDs to scan
`EMAIL_REPORTS_TO`	—	Comma-separated email recipients for reports
`EXPIRY_THRESHOLD_DAYS`	14	Days before expiry to flag and rotate
`ENABLE_ROTATION`	true	true = rotate; false = scan only
`STORAGE_BACKEND`	gcs	gcs or local
`GCS_BUCKET`	gcp-bucket-sa-keys-store	GCS bucket name for key storage
`ACS_CONNECTION_STRING`	—	Azure Communication Services connection string
`GMAIL_USER`	—	Gmail address for SMTP fallback
`GMAIL_APP_PASSWORD`	—	Gmail app-specific password for SMTP fallback

11 Deployment

Steps

1. **Create service account** — Cloud Run attached SA for Secret Manager access
2. **Assign IAM roles** — secretmanager.secretAccessor on host project; key admin roles on target projects
3. **Build container** — docker build or Cloud Build
4. **Deploy Cloud Run Job** — with bootstrap env vars and attached SA
5. **Schedule with Cloud Scheduler** — cron trigger (e.g. weekly on Mondays)

Key Deployment Commands

```
# Deploy the Cloud Run Job
gcloud run jobs create gcp-sa-key-rotation \
  --image=gcr.io/PROJECT_ID/gcp-sa-key-rotation:latest \
  --region=us-central1 \
  --service-account=cloud-run-rotator-sa@PROJECT_ID.iam.gserviceaccount.com \
  --set-env-vars="SECRET_MANAGER_PROJECT_ID=PROJECT_ID,SERVICE_ACCOUNT_SECRET_ID=SECRET_ID" \
  --max-retries=1 --task-timeout=600

# Schedule (every Monday 08:00 UTC)
gcloud scheduler jobs create http gcp-sa-key-rotation-schedule \
  --schedule="0 8 * * 1" \
  --uri="https://us-central1-run.googleapis.com/.../jobs/gcp-sa-key-rotation:run"
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

12 IAM & Security Model

Two-Layer Authentication

Layer	Service Account	Purpose
Layer 1 — Bootstrap	Cloud Run attached SA	Reads Secret Manager secrets only; no other permissions
Layer 2 — Operations	Main SA (from Secret Manager)	Performs all key operations across target projects

Required Roles

Role	Assigned To	Scope
`roles/secretmanager.secretAccessor`	Cloud Run SA	Host project
`roles/iam.serviceAccountKeyAdmin`	Main SA	Each target project
`roles/iam.serviceAccountViewer`	Main SA	Each target project
`roles/resourcemanager.projectViewer`	Main SA	Each target project
`roles/storage.objectAdmin`	Main SA	GCS bucket

> The Cloud Run Job has **minimal standing permissions**. All sensitive key operations are performed by the main SA — fetched at runtime, used only for the job duration.

13 Infrastructure Details

GCS Bucket: gcp-bucket-sa-keys-store

Property	Value
Purpose	Persistent storage for rotated SA key JSON credentials
Path structure	`service-account-keys/{project_id}/{sa_email}/{key_id}.json`
Retention	Latest key only — older files auto-deleted on new rotation
Access control	Uniform bucket-level access (no ACLs); IAM-only
Encryption	CMEK recommended for key material at rest

Cloud Run Job Settings

Setting	Value	Reason
CPU	1 vCPU	Key generation and API calls are not CPU-intensive
Memory	512 MiB	Sufficient for all in-memory operations
Task timeout	600s (10 min)	Accommodates large project scans + retry delays
Max retries	1	All operations are idempotent — safe to retry once
Schedule	`0 8 * * 1`	Every Monday 08:00 UTC (adjust per threshold)

14 Advantages

Security

- Private key never leaves your environment — zero exposure to GCP API surface
- All credentials in Secret Manager — no hardcoded secrets in code or containers
- Least-privilege architecture — minimal standing permissions for the Cloud Run Job
- Full audit trail via GCS object history, Cloud Audit Logs, and Excel reports

Scalability

- Scans any number of GCP projects in a single run
- Add new projects by updating a single Secret Manager secret — no redeployment

Reliability

- Per-SA error isolation — one failure does not block other service accounts
- Key validation with retry — confirms new key works despite GCP propagation delay
- Dual email path — ACS primary with Gmail SMTP fallback reduces alerting failures
- Zero-downtime rotation — old and new keys coexist during the handover window

Maintainability

- Config-driven — all parameters in Secret Manager, no code changes required to update config
- Modular design — cryptography, IAM, storage, and email fully decoupled
- Scan mode — provides full audit visibility without any mutations

15 Limitations

Limitation	Impact
No automatic deletion of old IAM keys	Old keys remain active in GCP IAM after rotation; manual cleanup required once applications are updated

Limitation	Impact
Manual key distribution	New keys are stored in GCS but not pushed to applications — a team handover process is required
No automatic application update	Applications must be manually updated with the new key JSON; restart and validation is a team responsibility
Only latest key per SA evaluated	If a SA has multiple keys, older ones are ignored and may expire silently
No rollback on validation failure	If key validation fails, the new key remains in GCP; manual cleanup or re-run needed
ACS email rate limits	Heavy volumes may trigger 429s; handled by Gmail fallback, but Gmail also has per-day send limits

16 Use Cases

- **Enterprise IAM governance** — enforce rotation policy across all GCP projects from one automated job
- **FinOps security automation** — reduce operational overhead in finance and billing platforms
- **Compliance requirements** — supports SOC 2, ISO 27001, and other frameworks requiring regular credential rotation and audit evidence
- **Multi-project GCP environments** — centralised rotation across dev, staging, and production without per-project tooling

17 Team Scope of Work & Key Distribution

Step 1 — Identify Rotated Keys

The job sends two reports after each run. Teams review them to identify service accounts with status **Rotated**:

- **HTML email** — colour-coded summary with metrics card and key details table
- **Excel attachment** — full detail including storage location, rotation timestamp, and validation result

Step 2 — Access Control

Team	GCS Bucket Access
DevOps Team	Full — read, write, delete
Ops Team (limited members)	Read only
Application Team	No direct access — receives keys via controlled handover

Step 3 — Key Retrieval from GCS

```
gsutil ls gs://gcp-bucket-sa-keys-store/service-account-
keys/PROJECT/sa@project.iam.gserviceaccount.com/
gsutil cp gs://gcp-bucket-sa-keys-store/.../KEY_ID.json ./
```

Step 4 — Distribution Flow

Step 5 — Application Team Responsibilities

1. Update the application configuration or secret store with the new SA key JSON
2. Restart or refresh any services holding the old credential in memory
3. Validate that application functionality and GCP access are fully restored

Security Guidelines

> **These rules apply to every member involved in key distribution:**
>
> - Access to the GCS bucket is governed strictly by IAM — do not grant access outside the approved list
> - Keys must be shared only via **secure, approved channels** (org-approved secrets manager or encrypted transfer)
> - **Never share keys via email attachments, Slack DMs, or any unencrypted medium**
> - **Never store keys locally** beyond the immediate update window
> - Delete locally downloaded key files immediately after updating the application

18 Operational Runbook

Execute Job Manually

```
gcloud run jobs execute gcp-sa-key-rotation \
  --region=us-central1 --project=finops-billing-central-prod
```

Check Execution Status

```
gcloud run jobs executions list \
  --job=gcp-sa-key-rotation --region=us-central1 \
  --project=finops-billing-central-prod
```

View Logs

```
gcloud logging read \
  'resource.type="cloud_run_job" AND resource.labels.job_name="gcp-sa-key-rotation"' \
  --limit=200 --project=finops-billing-central-prod
```

Run in Scan-Only Mode

```
ENABLE_ROTATION=false   # Set this secret value, then execute the job
```

Exit Codes

Code	Meaning
`0`	All scans and rotations completed successfully
`1`	One or more key rotations failed — check logs and Excel report

19 Final Outcome

This solution delivers a production-grade, enterprise-ready key rotation system

- **Secure key lifecycle management** — private key never leaves the environment; only the public cert reaches GCP
- **Fully automated rotation** — scheduled Cloud Run Job handles scan, rotate, validate, and report end-to-end
- **Enterprise-grade reporting** — colour-coded HTML email + full Excel attachment delivered to stakeholders after every run
- **Controlled team-based distribution** — IAM-gated GCS access with a defined DevOps → Ops → App Team handover workflow
- **Zero downtime operations** — old and new keys coexist in GCP IAM, ensuring no service disruption during rotation